

UNIT-1

Collection Framework

Why Collection Framework?

Before collection framework we used,

Variable: Variable is a name which is used to store the value. It holds only one value at a time.

Ex: `int a=10;`

Note:

Declaring more number of variables is not a right practice as the identification of variables is difficult if number of variables are large.

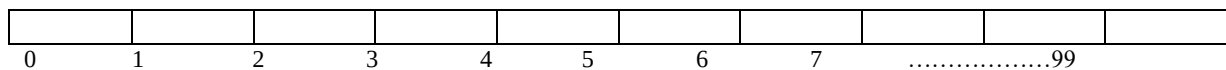
To avoid this, arrays were introduced.

Arrays:

Array is a collection of homogeneous datatypes.

Ex: `Student [] s=new Student[100];`

s



Limitations of arrays:

- Static memory location (size of array is fixed).
- Arrays allow only homogeneous datatypes.

Ex1: `s[0]=new Student();`

Ex2: `s[1]=new Student();`

Ex3: `s[2]=new Customer();`

If we execute the above examples, in Ex3 we get compile time error, Incompatible types.

- Ready-made methods are not available in arrays. It supports standard data structures.

Note: By using `Object Array` we can resolve the problems.

`Object [] o=new Object[100];`

`o[0]=new Student();`

`o[1]=new Customer();`

Collection:

A group of individual objects represented in a single entity.

What is Collection Framework?

It defines several classes and interfaces which can be used as a group of objects as a single entity.

In Java

1.Collection

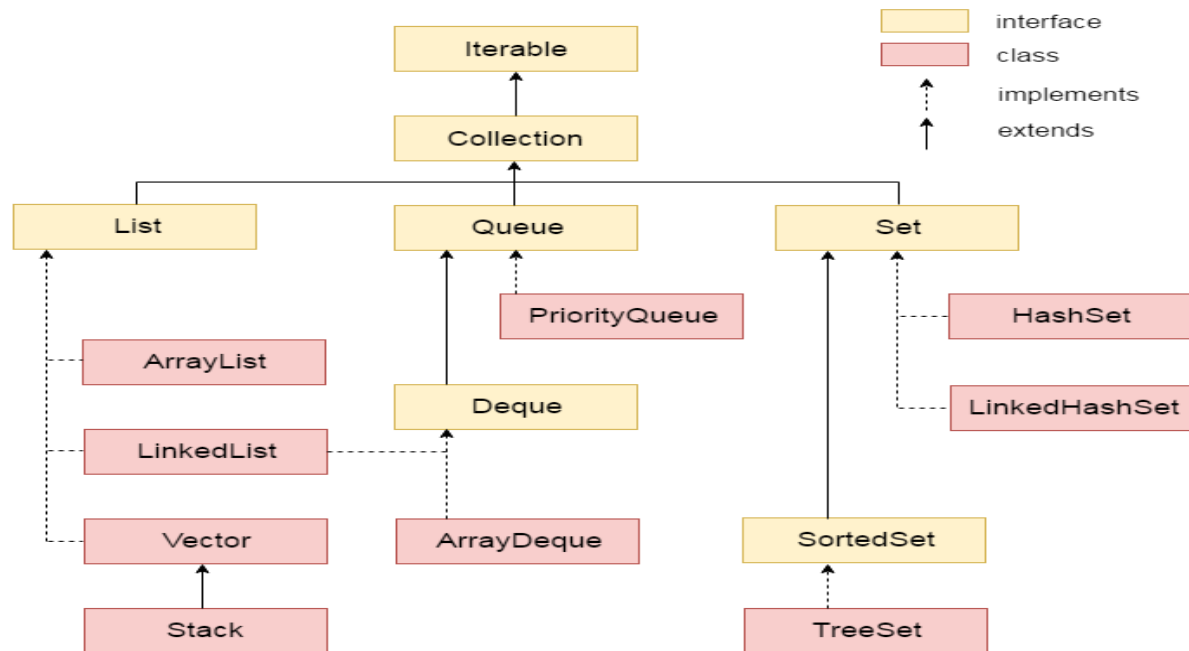
2.Collection Framework

- Collection Framework was introduced from JDK 1.2 version onwards.
- Collection framework is implemented in “java.util” package.

In C++

1.Container

2.STL(Standard Template Library)



Collection Interface

- To represent group of objects as a single entity we are using collection interface.
- Collection is the root interface of all classes and interfaces of CollectionFramework.
- It was introduced from JDK 1.2 version onwards.
- Collection interface defines the most common methods which are applicable for any Collection Object.
- There are many methods declared in the Collection interface. They are as follows:

No.	Method	Description
1	Public boolean add(Object element)	is used to insert an element in this collection.
2	public booleanaddAll(Collection c)	is used to insert the specified collection elements in the invoking collection.
3	public boolean remove(Object element)	is used to delete an element from this collection.
4	public booleanremoveAll(Collection c)	is used to delete all the elements of specified collection from the invoking collection.
5	public booleanretainAll(Collection c)	is used to delete all the elements of invoking collection except the specified collection.
6	public int size()	return the total number of elements in the collection.

7	public void clear()	removes the total no of element from the collection.
8	public boolean contains(Object element)	is used to search an element.
9	public boolean containsAll(Collection c)	is used to search the specified collection in this collection.
10	public Iterator iterator()	returns an iterator.
11	public Object[] toArray()	converts collection into array.
12	public boolean isEmpty()	checks if collection is empty.
13	public boolean equals(Object element)	matches two collection.
14	public int hashCode()	returns the hashcode number for collection.

Ex:

```
import java.util.*;
class ExOnCollection{
public static void mai(String[] args){
Collection c=new ArrayList();
c.add(10);
c.add(20);
c.add(30);
System.out.println(c);
}
}
```

Iterator interface

Iterator interface provides the facility of iterating the elements in forward direction only.

Methods of Iterator interface

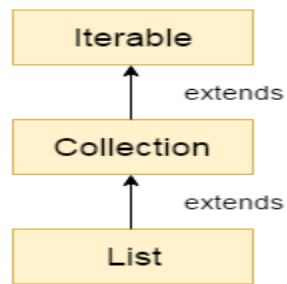
There are only three methods in the Iterator interface. They are:

No.	Method	Description
1	public boolean hasNext()	It returns true if iterator has more elements.
2	public Object next()	It returns the element and moves the cursor pointer to the next element.
3	public void remove()	It removes the last elements returned by the iterator. It is rarely used.

Note:

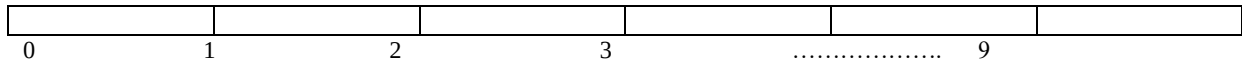
Collection interface doesn't contain any method to retrieve objects, there is no concrete class which implements collection class directly.

List Interface



- List is a child interface of collection.
- It was introduced from JDK 1.2 version onwards.
- If we want to represent a group of individual objects as a single entity where duplicates are allowed and insertion order is preserved we go for List.
- Insertion order is preserved using “index”.
- We can differentiate duplicates by using “index”.
- Insertion order is preserved (In which order you inserted, in the same order we retrieved).

index



Methods in List:

Sr.No.	Method & Description
1	void add(int index, Object obj) Inserts obj into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten.
2	boolean addAll(int index, Collection c) Inserts all elements of c into the invoking list at the index passed in the index. Any pre-existing elements at or beyond the point of insertion are shifted up. Thus, no elements are overwritten. Returns true if the invoking list changes and returns false otherwise.
3	Object get(int index) Returns the object stored at the specified index within the invoking collection.
4	int indexOf(Object obj)

	Returns the index of the first instance of obj in the invoking list. If obj is not an element of the list, .1 is returned.
5	<code>intlastIndexOf(Object obj)</code> Returns the index of the last instance of obj in the invoking list. If obj is not an element of the list, .1 is returned.
6	<code>ListIteratorlistIterator()</code> Returns an iterator to the start of the invoking list.
7	<code>ListIteratorlistIterator(int index)</code> Returns an iterator to the invoking list that begins at the specified index.
8	<code>Object remove(int index)</code> Removes the element at position index from the invoking list and returns the deleted element. The resulting list is compacted. That is, the indexes of subsequent elements are decremented by one.
9	<code>Object set(int index, Object obj)</code> Assigns obj to the location specified by index within the invoking list.
10	<code>List subList(int start, int end)</code> Returns a list that includes elements from start to end.1 in the invoking list. Elements in the returned list are also referenced by the invoking object.

Ex:

```
import java.util.List;
```

```
Import java.util.ArrayList;
```

```
Class ExOnList
```

```
{
```

```
public static void main(String[] args){
```

```
List l=new ArrayList();
```

```

l.add(10);

l.add(20);

l.add(30);

l.add(40);

System.out.println(l.indexOf(20));

L.add(20);

System.out.println(l.indexOf(20));

System.out.println(l.get(4));

System.out.println(l.set(4,50));

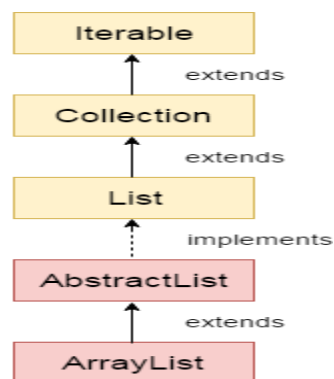
System.out.println(l);

}

}

```

ArrayList



- ArrayList is a child class of List interface.
- It is implemented on the underlying data structure that is “Resizable array” or “Growable Array”.
- Duplicates are not allowed.
- Insertion order is preserved.
- It allows null values.
- It allows random access because it implements RandomAccess Interface.
- It follows index to perform random access.
- It allows both homogeneous and heterogeneous objects.

Constructors of Java ArrayList

Constructor	Description
ArrayList()	It is used to build an empty array list.
ArrayList(Collection c)	It is used to build an array list that is initialized with the elements of the collection c.

ArrayList(int capacity)	It is used to build an array list that has the specified initial capacity.
-------------------------	--

Methods of Java ArrayList

Method	Description
void add(int index, Object element)	It is used to insert the specified element at the specified position index in a list.
boolean addAll(Collection c)	It is used to append all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's iterator.
void clear()	It is used to remove all of the elements from this list.
int lastIndexOf(Object o)	It is used to return the index in this list of the last occurrence of the specified element, or - 1 if the list does not contain this element.
Object[] toArray()	It is used to return an array containing all of the elements in this list in the correct order.
Object[] toArray(Object[] a)	It is used to return an array containing all of the elements in this list in the correct order.
boolean add(Object o)	It is used to append the specified element to the end of a list.
boolean addAll(int index, Collection c)	It is used to insert all of the elements in the specified collection into this list, starting at the specified position.
Object clone()	It is used to return a shallow copy of an ArrayList.
int indexOf(Object o)	It is used to return the index in this list of the first occurrence of the specified element, or - 1 if the List does not contain this element.
void trimToSize()	It is used to trim the capacity of this ArrayList instance to be the list's current size.

Java ArrayList Example

```
import java.util.*;
class TestCollection1{
    public static void main(String args[]){
        ArrayList<String> list=new ArrayList<String>();//Creating arraylist
        list.add("Ravi");//Adding object in arraylist
        list.add("Vijay");
        list.add("Ravi");
        list.add("Ajay");
        //Traversing list through Iterator
        Iterator itr=list.iterator();
        while(itr.hasNext()){
            System.out.println(itr.next());
        }
    }
}
```

Output:

```
Ravi
Vijay
Ravi
Ajay
```

Ways to iterate the elements of collection in java

There are two ways to traverse collection elements:

1. By Iterator interface.
2. By for-each loop.

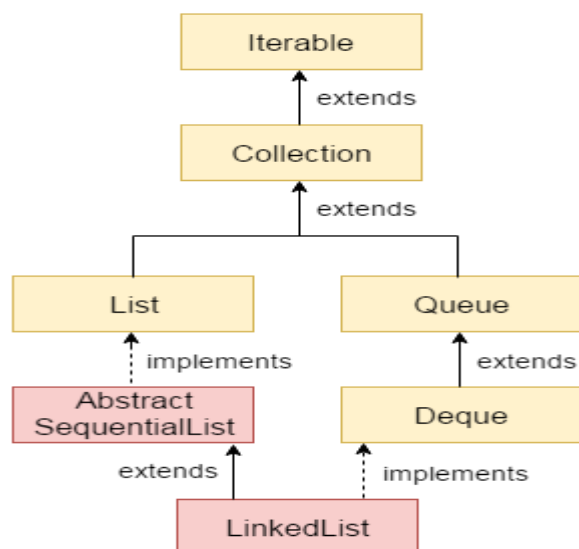
In the above example, we have seen traversing ArrayList by Iterator. Let's see the example to traverse ArrayList elements using for-each loop.

Iterating Collection through for-each loop

```
import java.util.*;
class TestCollection2{
public static void main(String args[]){
ArrayList<String> al=new ArrayList<String>();
al.add("Ravi");
al.add("Vijay");
al.add("Ravi");
al.add("Ajay");
for(String obj:al)
System.out.println(obj);
}
}
```

Ravi
Vijay
Ravi
Ajay

JavaLinkedList



- Underlying datastructure for LinkedList is double LinkedList.
 - Insertion order is preserved(we retrieve elements in insertion order).It doesnot allow to store key,valuepairs.
 - Duplicte are not allowed.
 - Heterogeneous elements or objects are allowed.
 - Null insertion is possible.
 - LinkedList implements “Serializable” and “Clonable” interfaces but not RandomAccess interface.So,it doesn't allows RandomAccess.
 - LinkedList is best choice if our frequent operation is insertion or deletion in the middle.
 - LinkedList is worst choice if our frequent operation is retrieval operation.
- In case of doubly linked list, we can add or remove elements from both side.

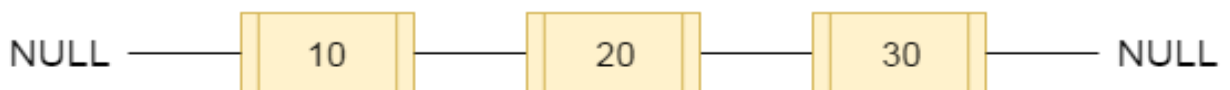


fig- doubly linked list

Java LinkedList class using doubly linked list

Constructors of Java LinkedList

Constructor	Description
LinkedList()	It is used to construct an empty list.
LinkedList(Collection c)	It is used to construct a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.

Methods of Java LinkedList

Method	Description
void add(int index, Object element)	It is used to insert the specified element at the specified position index in a list.
void addFirst(Object o)	It is used to insert the given element at the beginning of a list.
void addLast(Object o)	It is used to append the given element to the end of a list.
int size()	It is used to return the number of elements in a list
boolean add(Object o)	It is used to append the specified element to the end of a list.
boolean contains(Object o)	It is used to return true if the list contains a specified element.
boolean remove(Object o)	It is used to remove the first occurrence of the specified element in a list.
Object getFirst()	It is used to return the first element in a list.
Object getLast()	It is used to return the last element in a list.
int indexOf(Object o)	It is used to return the index in a list of the first occurrence of the specified element, or -1 if the list does not contain any element.
int lastIndexOf(Object o)	It is used to return the index in a list of the last occurrence of the specified element, or -1 if the list does not contain any element.

Java LinkedList Example

```
import java.util.*;

public class TestCollection7{
    public static void main(String args[]){
        LinkedList<String> al=new LinkedList<String>();
        al.add("Ravi");
        al.add("Vijay");
        al.add("Ravi");
        al.add("Ajay");

        Iterator<String> itr=al.iterator();
        while(itr.hasNext()){
            System.out.println(itr.next());
        }
    }
}
```

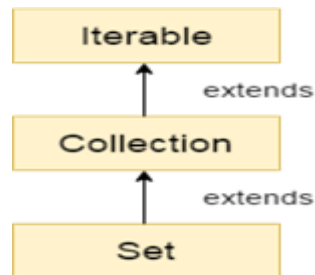
Difference between ArrayList and LinkedList:

Note: ArrayList and LinkedList both implements List interface and maintains insertion order.

ArrayList	LinkedList
1. ArrayList internally uses dynamic array to store elements.	1. LinkedList internally uses doubly linked list to store the elements.
2. Manipulation with ArrayList is slow because it internally uses array. If any element is removed from the array, all the	2. Manipulation with LinkedList is faster than ArrayList because it uses doubly linked list so no bit shifting is

bits are shifted in memory.	required in memory.
3.ArrayList class can act as a list only because it implements List only.	3.LinkedList can act as a list and queue both because it implements List and Dequeue interface.
4.ArrayList is better for storing and accessing data.	4.LinkedList is better for manipulating data.

Java Vector



- A Vector also stores objects similar to ArrayList, but vector is synchronized. It means even if several threads act on Vector object simultaneously, the result will be reliable.
- Vector implements a dynamic memory.
- Vector implements List Interface. Like an ArrayList it also maintains insertion order but it is rarely used in non-thread environment as it is synchronised and due to which gives poor performance in searching, adding, deleting and update of its elements.
- Insertion order is preserved.
- It allows RandomAccess of objects.
- It doesn't allow key, value pairs to store.
- Duplicate elements are allowed.
- "NULL" values are allowed.
- It provides thread safe.

Following is the list of constructors provided by the vector class.

Sr.No.	Constructor & Description
1	Vector() This constructor creates a default vector, which has an initial size of 10.
2	Vector(int size) This constructor accepts an argument that equals to the required size, and creates a vector whose initial capacity is specified by size.
3	Vector(int size, int incr) This constructor creates a vector whose initial capacity is specified by size and whose increment is specified by incr. The increment specifies the number of elements to allocate each time that a vector is resized upward.
4	Vector(Collection c) This constructor creates a vector that contains the elements of collection c.

Apart from the methods inherited from its parent classes, Vector defines the following methods –

Sr.No.	Method & Description
1	void add(int index, Object element) Inserts the specified element at the specified position in this Vector.
2	boolean add(Object o) Appends the specified element to the end of this Vector.
3	boolean addAll(Collection c) Appends all of the elements in the specified Collection to the end of this Vector, in the order that they are returned by the specified Collection's Iterator.
4	boolean addAll(int index, Collection c) Inserts all of the elements in in the specified Collection into this Vector at the specified position.
5	void addElement(Object obj) Adds the specified component to the end of this vector, increasing its size by one.
6	int capacity() Returns the current capacity of this vector.
7	void clear() Removes all of the elements from this vector.
8	Object clone() Returns a clone of this vector.
9	boolean contains(Object elem) Tests if the specified object is a component in this vector.
10	boolean containsAll(Collection c) Returns true if this vector contains all of the elements in the specified Collection.
11	void copyInto(Object[] anArray) Copies the components of this vector into the specified array.
12	Object elementAt(int index) Returns the component at the specified index.

13	Enumeration elements() Returns an enumeration of the components of this vector.
14	void ensureCapacity(int minCapacity) Increases the capacity of this vector, if necessary, to ensure that it can hold at least the number of components specified by the minimum capacity argument.
15	boolean equals(Object o) Compares the specified Object with this vector for equality.
16	Object firstElement() Returns the first component (the item at index 0) of this vector.
17	Object get(int index) Returns the element at the specified position in this vector.
18	int hashCode() Returns the hash code value for this vector.
19	int indexOf(Object elem) Searches for the first occurrence of the given argument, testing for equality using the equals method.
20	int indexOf(Object elem, int index) Searches for the first occurrence of the given argument, beginning the search at index, and testing for equality using the equals method.
21	void insertElementAt(Object obj, int index) Inserts the specified object as a component in this vector at the specified index.
22	boolean isEmpty() Tests if this vector has no components.
23	Object lastElement() Returns the last component of the vector.
24	int lastIndexOf(Object elem) Returns the index of the last occurrence of the specified object in this vector.
25	int lastIndexOf(Object elem, int index)

	Searches backwards for the specified object, starting from the specified index, and returns an index to it.
26	Object remove(int index) Removes the element at the specified position in this vector.
27	boolean remove(Object o) Removes the first occurrence of the specified element in this vector, If the vector does not contain the element, it is unchanged.
28	boolean removeAll(Collection c) Removes from this vector all of its elements that are contained in the specified Collection.
29	void removeAllElements() Removes all components from this vector and sets its size to zero.
30	boolean removeElement(Object obj) Removes the first (lowest-indexed) occurrence of the argument from this vector.
31	void removeElementAt(int index) removeElementAt(int index).
32	protected void removeRange(int fromIndex, int toIndex) Removes from this List all of the elements whose index is between fromIndex, inclusive and toIndex, exclusive.
33	boolean retainAll(Collection c) Retains only the elements in this vector that are contained in the specified Collection.
34	Object set(int index, Object element) Replaces the element at the specified position in this vector with the specified element.
35	void setElementAt(Object obj, int index) Sets the component at the specified index of this vector to be the specified object.
36	void setSize(int newSize) Sets the size of this vector.
37	int size() Returns the number of components in this vector.

38	List subList(int fromIndex, int toIndex) Returns a view of the portion of this List between fromIndex, inclusive, and toIndex, exclusive.
39	Object[] toArray() Returns an array containing all of the elements in this vector in the correct order.
40	Object[] toArray(Object[] a) Returns an array containing all of the elements in this vector in the correct order; the runtime type of the returned array is that of the specified array.
41	String toString() Returns a string representation of this vector, containing the String representation of each element.
42	void trimToSize() Trims the capacity of this vector to be the vector's current size.

Ex:

```
import java.util.*;
public class PrintElements
{
    public static void main(String args[])
    {
        Vector vect1 = new Vector();

        vect1.add("Jyostna");
        vect1.add(100);
        vect1.add(10.5);
        vect1.add(new Date());

        Enumeration e = vect1.elements();

        System.out.println("Vector Elements:");

        while(e.hasMoreElements())
        {
            System.out.println(e.nextElement());
        }
    }
}
```

Enumeration interface methods:

The Enumeration interface defines the methods by which you can enumerate (obtain one at a time) the elements in a collection of objects.

This legacy interface has been superceded by Iterator. Although not deprecated, Enumeration is considered obsolete for new code. However, it is used by several methods defined by the legacy classes such as Vector and Properties, is used by several other API classes, and is currently in widespread use in application code.

Sr.No.	Method & Description
--------	----------------------

1	boolean hasMoreElements() When implemented, it must return true while there are still more elements to extract, and false when all the elements have been enumerated.
2	Object nextElement() This returns the next object in the enumeration as a generic Object reference.

Difference between ArrayList and Vector:

<u>ArrayList</u>	<u>Vector</u>
1.ArrayList is not synchronized.	1.Vector is synchronized.
2.ArrayList increments 50% of current arraysize if number of elements exceeds from its capacity.	2.Vector increments 100% means doubles the array size if total number of elemen exceeds than its capacity.
3.ArrayList is not a legacy class.it was introduced in JDK 1.2.	3.Vector is a legacy class.
4.ArrayList is fast because it is not synchronized.	4.Vector is slow because it is synchronized.
5.ArrayList use Iterator interface to traverse the elements.	5.Vector uses Enumeration interface to traverse the elements,but it can use iterator also.

Java Stack

- Stack is a subclass of Vector that implements a standard last-in, first-out stack.
- Stack only defines the default constructor, which creates an empty stack. Stack includes all the methods defined by Vector, and adds several of its own.

Sr.No.	Method & Description
1	boolean empty() Tests if this stack is empty. Returns true if the stack is empty, and returns false if the stack contains elements.
2	Object peek() Returns the element on the top of the stack, but does not remove it.
3	Object pop() Returns the element on the top of the stack, removing it in the process.
4	Object push(Object element) Pushes the element onto the stack. Element is also returned.
5	int search(Object element) Searches for element in the stack. If found, its offset from the top of the stack is returned. Otherwise, .1 is returned.

Ex:

```

import java.util.*;

public class StackDemo {

    static void showpush(Stack st, int a) {
        st.push(new Integer(a));
        System.out.println("push(" + a + ")");
        System.out.println("stack: " + st);
    }

    static void showpop(Stack st) {
        System.out.print("pop -> ");
        Integer a = (Integer) st.pop();
        System.out.println(a);
        System.out.println("stack: " + st);
    }

    public static void main(String args[]) {
        Stack st = new Stack();
        System.out.println("stack: " + st);
        showpush(st, 42);
        showpush(st, 66);
        showpush(st, 99);
        showpop(st);
        showpop(st);
        showpop(st);
        try {
            showpop(st);
        } catch (EmptyStackException e) {
            System.out.println("empty stack");
        }
    }
}

```

Set Interface

- Set is child interface of Collection interface. It was introduced from JDK 1.2 version onwards.
- A set represents a group of elements arranged just like an array.
- The set will grow dynamically when the elements are stored into it.
- A set will not allow duplicate elements. If we try to pass the same element that already exists in the Set, then it is not stored into the Set.

Methods of Set Interface:

Sr.No.	Method & Description
1	add() Adds an object to the collection.
2	clear() Removes all objects from the collection.
3	contains() Returns true if a specified object is an element within the collection.
4	isEmpty() Returns true if the collection has no elements.
5	iterator() Returns an Iterator object for the collection, which may be used to retrieve an object.
6	remove() Removes a specified object from the collection.
7	size() Returns the number of elements in the collection.

Example

Set has its implementation in various classes like HashSet, TreeSet, LinkedHashSet. Following is an example to explain Set functionality

```
import java.util.*;

public class SetDemo {

    public static void main(String args[]) {

        int count[] = {34, 22, 10, 60, 30, 22};

        Set<Integer> set = new HashSet<Integer>();

        try {

            for(int i = 0; i < 5; i++) {

                set.add(count[i]);

            }

            System.out.println(set);

            TreeSet sortedSet = new TreeSet<Integer>(set);

            System.out.println("The sorted list is:");
```

```

System.out.println(sortedSet);

System.out.println("The First element of the set is: "+ (Integer)sortedSet.first());

System.out.println("The last element of the set is: "+ (Integer)sortedSet.last());

    }

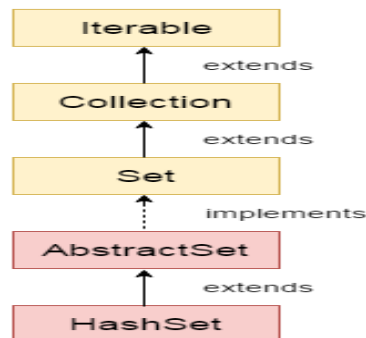
catch(Exception e) {}

}

}

```

Java HashSet Class



- HashSet is the child class of Set Interface .It was introduced from JDK 1.2 version onwards.
- A HashSet represents a set of elements.
- It doesn't maintain any kind of order of its elements.
- It doesn't allow the duplicate values to be stored.
- It doesn't allow RandomAccess.

Constructors of Java HashSet class:

Constructor	Description
HashSet()	It is used to construct a default HashSet.
HashSet(Collection c)	It is used to initialize the hash set by using the elements of the collection c.
HashSet(int capacity)	It is used to initialize the capacity of the hash set to the given integer value capacity. The capacity grows automatically as elements are added to the HashSt.

Methods of Java HashSet class:

Method	Description
void clear()	It is used to remove all of the elements from this set.
boolean contains(Object o)	It is used to return true if this set contains the specified element.
boolean add(Object o)	It is used to add the specified element to this set if it is not already present.
boolean isEmpty()	It is used to return true if this set contains no elements.
boolean remove(Object o)	It is used to remove the specified element from this set if it is present.
Object clone()	It is used to return a shallow copy of this HashSet instance: the elements themselves are not cloned.
Iterator iterator()	It is used to return an iterator over the elements in this set.
int size()	It is used to return the number of elements in this set.

Java HashSet Example

```
import java.util.*;

class TestCollection9{

public static void main(String args[]){

    //Creating HashSet and adding elements

    HashSet<String> set=new HashSet<String>();

    set.add("Ravi");

    set.add("Vijay");

    set.add("Ravi");

    set.add("Ajay");

    //Traversing elements

    Iterator<String> itr=set.iterator();

    while(itr.hasNext()){

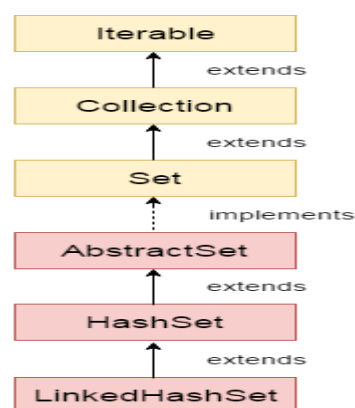
        System.out.println(itr.next());

    }

}

}
```

Java LinkedHashSet class



- It is a child class of HashSet class and it doesn't contain any additional methods on its own.
- It contains unique elements.
- It allows NULL values.
- It maintains insertion order.
- The underline data structure for LinkedHashSet is Doubly linked list.

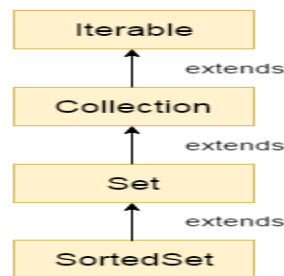
Constructors of Java LinkedHashSet class

Constructor	Description
HashSet()	It is used to construct a default HashSet.
HashSet(Collection c)	It is used to initialize the hash set by using the elements of the collection c.
LinkedHashSet(int capacity)	It is used initialize the capacity of the linkedhashset to the given integer value capacity.
LinkedHashSet(int capacity, float fillRatio)	It is used to initialize both the capacity and the fill ratio (also called load capacity) of the hash set from its argument.

Example of LinkedHashSet class:

```
import java.util.*;
class TestCollection10{
    public static void main(String args[]){
        LinkedHashSet<String> al=new LinkedHashSet<String>();
        al.add("Ravi");
        al.add("Vijay");
        al.add("Ravi");
        al.add("Ajay");
        Iterator<String> itr=al.iterator();
        while(itr.hasNext()){
            System.out.println(itr.next());
        }
    }
}
```

Java SortedSet Interface



- SortedSet is child interface of Set interface. It was introduced from JDK 1.2 version onwards.
- It declares the behaviour of a sorted set in ascending order.

Note: In SortedSet several methods may rise exceptions like:

- NoSuchElementException, when an object is not available in the sorted set.
- ClassCastException is generated when object is incompatible with SortedSet.

Methods of Sorted Set interface:

- **comparator()** : Returns the comparator used to order the elements in this set, or null if this set uses the natural ordering of its elements.
- **first()** : Returns the first (lowest) element currently in this set.
- **headSet(E toElement)** : Returns a view of the portion of this set whose elements are strictly less than toElement.
- **last()** : Returns the last (highest) element currently in this set.
- **subSet(E fromElement, E toElement)** : Returns a view of the portion of this set whose elements range from fromElement, inclusive, to toElement, exclusive.
- **tailSet(E fromElement)** : Returns a view of the portion of this set whose elements are greater than or equal to fromElement.

Ex:

```
import java.util.SortedSet;
import java.util.TreeSet;
public class Main
{
    public static void main(String[] args)
    {
        // Create a TreeSet and inserting elements
```

```

SortedSet<String> sites = new TreeSet<>();
sites.add("practice");
sites.add("geeksforgeeks");
sites.add("quiz");
sites.add("code");

System.out.println("Sorted Set: " + sites);
System.out.println("First: " + sites.first());
System.out.println("Last: " + sites.last());
// Getting elements before quiz (Excluding) in a sortedSet
SortedSet<String> beforeQuiz = sites.headSet("quiz");
System.out.println(beforeQuiz);
// A Java program to demonstrate working of SortedSet
import java.util.SortedSet;
import java.util.TreeSet;

public class Main
{
    public static void main(String[] args)
    {
        // Create a TreeSet and inserting elements
        SortedSet<String> sites = new TreeSet<>();
        sites.add("practice");
        sites.add("geeksforgeeks");
        sites.add("quiz");
        sites.add("code");

        System.out.println("Sorted Set: " + sites);
        System.out.println("First: " + sites.first());
        System.out.println("Last: " + sites.last());

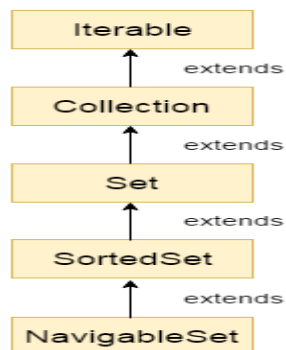
        // Getting elements before quiz (Excluding) in a sortedSet
        SortedSet<String> beforeQuiz = sites.headSet("quiz");
        System.out.println(beforeQuiz);

        // Getting elements between code (Including) and
        // practice (Excluding)
        SortedSet<String> betweenCodeAndQuiz =
            sites.subSet("code", "practice");
        System.out.println(betweenCodeAndQuiz);

        // Getting elements after code (Including)
        SortedSet<String> afterCode = sites.tailSet("code");
        System.out.println(afterCode);
    }
}

```

Java NavigableSet Interface



- It is a child interface of SortedSet. It is available from JDK 1.6 version onwards.
- It is accessed in both ascending order and descending order.
- Performance of ascending order operation is faster than descending order.

Methods of NavigableSet

Lower(E e) : Returns the greatest element in this set which is less than the given element or NULL if there is no such element.
 Floor(E e) : Returns the greatest element in this set which is less than or equal to given element or NULL if there is no such element.
 Ceiling(E e) : Returns the least element in this set which is greater than or equal to given element or NULL if there is no such element.
 Higher(E e) : Returns the least element in this set which is greater than the given element or NULL if there is no such element.
 pollFirst() : Retrieve and remove the first least element. Or return null if there is no such element.
 pollLast() : Retrieve and remove the last highest element. Or return null if there is no such element.

```
import java.util.NavigableSet;

import java.util.TreeSet;

public class hashset
{
    public static void main(String[] args)
    {
        NavigableSet<Integer> ns = new TreeSet<>();

        ns.add(0);

        ns.add(1);

        ns.add(2);

        ns.add(3);

        ns.add(4);

        ns.add(5);

        ns.add(6);

        // Get a reverse view of the navigable set
        NavigableSet<Integer> reverseNs = ns.descendingSet();

        // Print the normal and reverse views
        System.out.println("Normal order: " + ns);

        System.out.println("Reverse order: " + reverseNs);

        NavigableSet<Integer> threeOrMore = ns.tailSet(3, true);

        // A Java program to demonstrate working of SortedSet

import java.util.NavigableSet;

import java.util.TreeSet;

public class hashset
{
    public static void main(String[] args)
    {
        NavigableSet<Integer> ns = new TreeSet<>();
```

```

ns.add(0);

ns.add(1);

ns.add(2);

ns.add(3);

ns.add(4);

ns.add(5);

ns.add(6);

// Get a reverse view of the navigable set
NavigableSet<Integer> reverseNs = ns.descendingSet();

// Print the normal and reverse views

System.out.println("Normal order: " + ns);

System.out.println("Reverse order: " + reverseNs);

NavigableSet<Integer> threeOrMore = ns.tailSet(3, true);

System.out.println("3 or more: " + threeOrMore);

System.out.println("lower(3): " + ns.lower(3));

System.out.println("floor(3): " + ns.floor(3));

System.out.println("higher(3): " + ns.higher(3));

System.out.println("ceiling(3): " + ns.ceiling(3));

System.out.println("pollFirst(): " + ns.pollFirst());

System.out.println("Navigable Set: " + ns);

System.out.println("pollLast(): " + ns.pollLast());

System.out.println("Navigable Set: " + ns);

System.out.println("pollFirst(): " + ns.pollFirst());

System.out.println("Navigable Set: " + ns);

System.out.println("pollFirst(): " + ns.pollFirst());

System.out.println("Navigable Set: " + ns);

System.out.println("pollFirst(): " + ns.pollFirst());

System.out.println("Navigable Set: " + ns);

System.out.println("pollFirst(): " + ns.pollFirst());

System.out.println("pollLast(): " + ns.pollLast());

}

}

```

Output:

Normal order: [0, 1, 2, 3, 4, 5, 6]

Reverse order: [6, 5, 4, 3, 2, 1, 0]

3 or more: [3, 4, 5, 6]

lower(3): 2

floor(3): 3

higher(3): 4

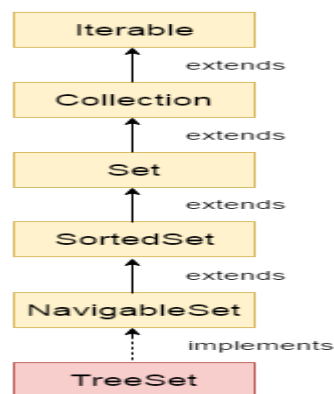
ceiling(3): 3

pollFirst(): 0

Navigable Set: [1, 2, 3, 4, 5, 6]

pollLast(): 6

Java TreeSet class



- TreeSet class is a child class of NavigableSet interface. It was introduced from JDK 1.2 version onwards.
- TreeSet is similar to HashSet except that it sorts the elements in the ascending order while HashSet doesn't maintain any order.
- It doesn't allow Null values.

Access and retrieval time are quite fast which makes TreeSet an excellent choice when storing large amounts of stored information that must be found very quickly.

Constructors of Java TreeSet class

Constructor	Description
TreeSet()	It is used to construct an empty tree set that will be sorted in an ascending order according to the natural order of the tree set.
TreeSet(Collection c)	It is used to build a new tree set that contains the elements of the collection c.
TreeSet(Comparator comp)	It is used to construct an empty tree set that will be sorted according to given comparator.
TreeSet(SortedSet ss)	It is used to build a TreeSet that contains the elements of the given SortedSet.

Methods of Java TreeSet class

Method	Description
--------	-------------

boolean addAll(Collection c)	It is used to add all of the elements in the specified collection to this set.
boolean contains(Object o)	It is used to return true if this set contains the specified element.
boolean isEmpty()	It is used to return true if this set contains no elements.
boolean remove(Object o)	It is used to remove the specified element from this set if it is present.
void add(Object o)	It is used to add the specified element to this set if it is not already present.
void clear()	It is used to remove all of the elements from this set.
Object clone()	It is used to return a shallow copy of this TreeSet instance.
Object first()	It is used to return the first (lowest) element currently in this sorted set.
Object last()	It is used to return the last (highest) element currently in this sorted set.
int size()	It is used to return the number of elements in this set.

Java TreeSet Example

```
import java.util.*;
class TestCollection11{
public static void main(String args[]){
    //Creating and adding elements
    TreeSet<String> al=new TreeSet<String>();
    al.add("Ravi");
    al.add("Vijay");
    al.add("Ravi");
    al.add("Ajay");
    //Traversing elements
    Iterator<String> itr=al.iterator();
    while(itr.hasNext()){
        System.out.println(itr.next());
    }
}
```

Output:

Ajay

Ravi

Vijay

Java Queue Interface

- Queue is an interface that extends Collection Interface. It was introduced from JDK 1.5 version onwards.
- It follows First In First Out order.
- It is available in java.util package.

Methods of Java Queue Interface

Method	Description
boolean add(object)	It is used to insert the specified element into this queue and return true upon success.
boolean offer(object)	It is used to insert the specified element into this queue.
Object remove()	It is used to retrieve and removes the head of this queue.

Object poll()	It is used to retrieves and removes the head of this queue, or returns null if this queue is empty.
Object element()	It is used to retrieves, but does not remove, the head of this queue.
Object peek()	It is used to retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.

Operations on Queue:

Type of Operation	Throws Exception	Returns Special value
insert	add(Object o)	offer(Object o)
remove	remove()	poll()
examine	element()	peek()

Ex:

```
import java.util.*;
class ExOnQueue{
public static void main(String[] args){
Queue q=new PriorityQueue();
q.add(10);
q.add(20);
q.add(30);
for(Object o: q)
System.out.println(o);
}
}
```

Java PriorityQueue class

- It is a child class of Queue Interface. It is introduced from JDK 1.5 version onwards.
- The underlying data structure for PriorityQueue is PriorityHeap.
- The PriorityQueue size is unbounded. We may give initial capacity of PriorityQueue at the time of its creation.
- It is not thread safe.

It doesn't order the objects in FIFO order. **Java PriorityQueue Example**

```
import java.util.*;
class TestCollection12{
public static void main(String args[]){
PriorityQueue<String> queue=new PriorityQueue<String>();
queue.add("Amit");
queue.add("Vijay");
queue.add("Karan");
queue.add("Jai");
queue.add("Rahul");
System.out.println("head:"+queue.element());
System.out.println("head:"+queue.peek());
System.out.println("iterating the queue elements:");
Iterator itr=queue.iterator();
```

```

while(itr.hasNext()){
System.out.println(itr.next());
}
queue.remove();
queue.poll();
System.out.println("after removing two elements:");
Iterator<String> itr2=queue.iterator();
while(itr2.hasNext()){
System.out.println(itr2.next());
}
}
}

```

Output:

head:Amit

head:Amit

iterating the queue elements:

Amit

Jai

Karan

Vijay

Rahul

after removing two elements:

Karan

Rahul

Vijay

Java Comparable interface:

Java Comparable interface is used to order the objects of user-defined class. This interface is found in java.lang package and contains only one method named compareTo(Object). It provides a single sorting sequence only i.e. you can sort the elements on the basis of a single data member only. For example, it may be rollno, name, age or anything else.

compareTo Java Comparable interface (Object obj) method

public int compareTo(Object obj): is used to compare the current object with the specified object. We can sort the elements of:

1. String objects

2. Wrapper class objects

3. User-defined class objects.

Difference between Comparable and Comparator:

Comparable and Comparator both are interfaces and can be used to sort collection elements.

But there are many differences between Comparable and Comparator interfaces that are given below.

Comparable	Comparator
1) Comparable provides single sorting sequence . In other words, we can sort the collection on the basis of single element such as id or name or price etc.	Comparator provides multiple sorting sequence . In other words, we can sort the collection on the basis of multiple elements such as id, name and price etc.
2) Comparable affects the original class i.e. actual class is modified.	Comparator doesn't affect the original class i.e. actual class is not modified.
3) Comparable provides compareTo() method to sort elements.	Comparator provides compare() method to sort elements.
4) Comparable is found in java.lang package.	Comparator is found in java.util package.
5) We can sort the list elements of Comparable type by Collections.sort(List) method.	We can sort the list elements of Comparator type by Collections.sort(List, Comparator) method.

Java Map Interface

- A map contains values on the basis of key i.e. key and value pair. Each key and value pair is known as an entry. Map contains only unique keys.
- Map is useful if you have to search, update or delete elements on the basis of key.

Java Map Hierarchy

- There are two interfaces for implementing Map in java: Map and SortedMap, and three classes: HashMap, LinkedHashMap and TreeMap. The hierarchy of Java Map is given below:

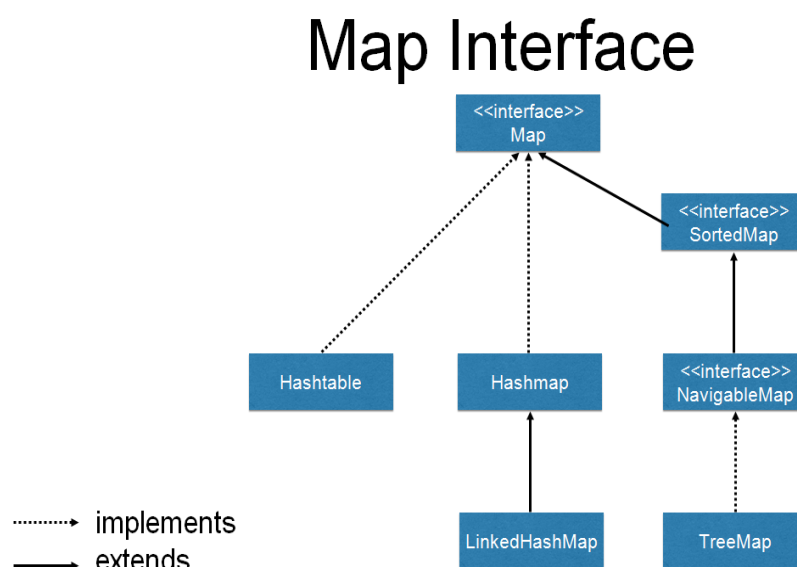


Fig: Java Map Hierarchy

- Map doesn't allow duplicate keys, but you can have duplicate values. HashMap and LinkedHashMap allows null keys and values but TreeMap doesn't allow any null key or value.
- Map can't be traversed so you need to convert it into Set using keySet() or entrySet() method.
- HashMap - HashMap is the implementation of Map but it doesn't maintain any order.
- LinkedHashMap-LinkedHashMap is the implementation of Map, it inherits HashMap class. It maintains insertion order.
- TreeMap-TreeMap is the implementation of Map and SortedMap, it maintains ascending order.

Useful methods of Map interface:

Method	Description
Object put(Object key, Object value)	It is used to insert an entry in this map.
void putAll(Map map)	It is used to insert the specified map in this map.
Object remove(Object key)	It is used to delete an entry for the specified key.
Object get(Object key)	It is used to return the value for the specified key.
boolean containsKey(Object key)	It is used to search the specified key from this map.
Set keySet()	It is used to return the Set view containing all keys.
Set entrySet()	It is used to return the Set view containing all keys and values

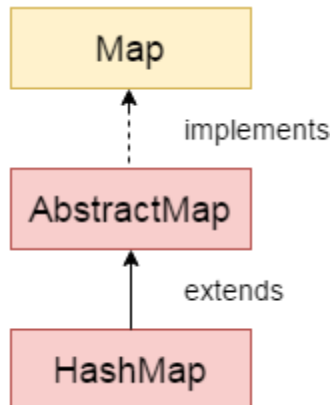
Map.Entry Interface

Entry is the sub interface of Map. So we will be accessed it by Map.Entry name. It provides methods to get key and value.

Methods of Map.Entry interface

Method	Description
Object getKey()	It is used to obtain key.
Object getValue()	It is used to obtain value.

Java HashMap class



Java HashMap class implements the map interface by using a hashtable. It inherits AbstractMap class and implements Map interface.

The important points about Java HashMap class are:

- A HashMap contains values based on the key.
- It contains only unique elements.
- It may have one null key and multiple null values.
- It maintains no order.

Hierarchy of HashMap class

As shown in the above figure, HashMap class extends AbstractMap class and implements Map interface.

HashMap class declaration

Let's see the declaration for java.util.HashMap class.

1. **public class** HashMap<K,V> **extends** AbstractMap<K,V> **implements** Map<K,V>, Cloneable, Serializable

HashMap class Parameters

Let's see the Parameters for java.util.HashMap class.

- **K**: It is the type of keys maintained by this map.
- **V**: It is the type of mapped values.

Constructors of Java HashMap class

Constructor	Description
HashMap()	It is used to construct a default HashMap.
HashMap(Map m)	It is used to initialize the hash map by using the elements of the given Map object m.
HashMap(int capacity)	It is used to initialize the capacity of the hash map to the given integer value, capacity.
HashMap(int capacity, float fillRatio)	It is used to initialize both the capacity and fill ratio of the hash map by using its arguments.

Methods of Java HashMap class

Method	Description
void clear()	It is used to remove all of the mappings from this map.
boolean containsKey(Object key)	It is used to return true if this map contains a mapping for the specified key.
boolean containsValue(Object value)	It is used to return true if this map maps one or more keys to the specified value.
boolean isEmpty()	It is used to return true if this map contains no key-value mappings.
Object clone()	It is used to return a shallow copy of this HashMap instance: the keys and values themselves are not cloned.
Set entrySet()	It is used to return a collection view of the mappings contained in this map.
Set keySet()	It is used to return a set view of the keys contained in this map.

Object put(Object key, Object value)	It is used to associate the specified value with the specified key in this map.
int size()	It is used to return the
Collection values()	It is used to return a collection view of the values contained in this map.

Java Map Example: Generic (New Style)

```
import java.util.*;

class MapInterfaceExample{

    public static void main(String args[]){

        Map<Integer,String> map=new HashMap<Integer,String>();

        map.put(100,"Amit");
        map.put(101,"Vijay");
        map.put(102,"Rahul");

        for(Map.Entry m:map.entrySet()){

            System.out.println(m.getKey()+" "+m.getValue());

        }

    }

}
```

Output:
102 Rahul
100 Amit
101 Vijay

Java Map Example: Non-Generic (Old Style)

```
//Non-generic

import java.util.*;

public class MapExample1 {

    public static void main(String[] args) {

        Map map=new HashMap();

        //Adding elements to map

        map.put(1,"Amit");

        map.put(5,"Rahul");

        map.put(2,"Jai");

        map.put(6,"Amit");

    }

}
```

```
//Traversing Map

Set set=map.entrySet();//Converting to Set so that we can traverse

Iterator itr=set.iterator();

while(itr.hasNext()){

    //Converting to Map.Entry so that we can get key and value separately

    Map.Entry entry=(Map.Entry)itr.next();

    System.out.println(entry.getKey()+" "+entry.getValue());

}

}

}
Output:
1 Amit
2 Jai
5 Rahul
6 Amit
```

Java HashMap Example

```
import java.util.*;

class TestCollection13{

    public static void main(String args[]){

        HashMap<Integer,String> hm=new HashMap<Integer,String>();

        hm.put(100,"Amit");

        hm.put(101,"Vijay");

        hm.put(102,"Rahul");

        hm.remove(102);

        for(Map.Entry m:hm.entrySet()){

            System.out.println(m.getKey()+" "+m.getValue());

        }

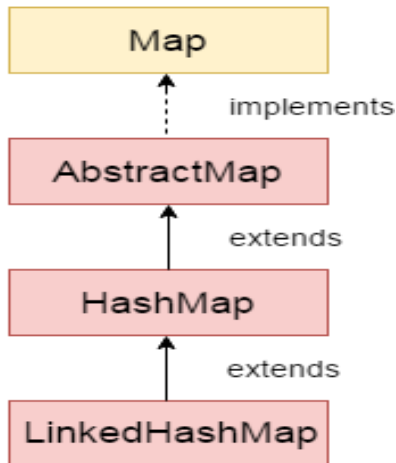
    }

} Test it Now
Output:102 Rahul
        100 Amit
        101 Vijay
```

Difference between HashSet and HashMap:

HashSet contains only values whereas HashMap contains entry(key and value).

Java LinkedHashMap class



Java LinkedHashMap class is Hash table and Linked list implementation of the Map interface, with predictable iteration order. It inherits HashMap class and implements the Map interface. The important points about Java LinkedHashMap class are:

- A LinkedHashMap contains values based on the key.
- It contains only unique elements.
- It may have one null key and multiple null values.
- It is same as HashMap instead maintains insertion order.

LinkedHashMap class declaration

Let's see the declaration for java.util.LinkedHashMap class.

1. **public class** LinkedHashMap<K,V> **extends** HashMap<K,V> **implements** Map<K,V>

LinkedHashMap class Parameters

Let's see the Parameters for java.util.LinkedHashMap class.

- **K**: It is the type of keys maintained by this map.
- **V**: It is the type of mapped values.

Constructors of Java LinkedHashMap class

LinkedHashMap()-It is used to construct a default LinkedHashMap.

LinkedHashMap(int capacity)- It is used to initialize a LinkedHashMap with the given capacity.

LinkedHashMap(int capacity, float fillRatio)- It is used to initialize both the capacity and the fillRatio.

LinkedHashMap(Map m).- It is used to initialize the LinkedHashMap with the elements from the given Map class m.

Methods of Java LinkedHashMap class

Object get(Object key)- It is used to return the value to which this map maps the specified key.

void clear()-It is used to remove all mappings from this map.

boolean containsKey(Object key)- It is used to return true if this map maps one or more keys to the specified value.

Java LinkedHashMap Example

```
import java.util.*;
```

```
class TestCollection14{
```

```
    public static void main(String args[]){
```

```
        LinkedHashMap<Integer,String> hm=new LinkedHashMap<Integer,String>();
```

```

    hm.put(100,"Amit");

    hm.put(101,"Vijay");

    hm.put(102,"Rahul");

    for(Map.Entry m:hm.entrySet()){

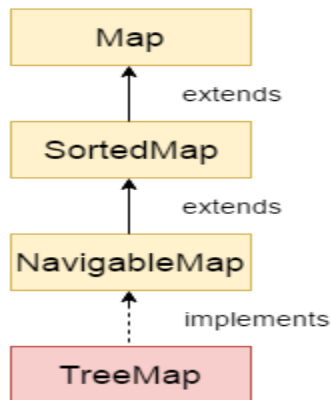
        System.out.println(m.getKey()+" "+m.getValue());

    }

}
}
}

```

Java TreeMap class



Java TreeMap class implements the Map interface by using a tree. It provides an efficient means of storing key/value pairs in sorted order.

The important points about Java TreeMap class are:

- A TreeMap contains values based on the key. It implements the NavigableMap interface and extends AbstractMap class.
- It contains only unique elements.
- It cannot have null key but can have multiple null values.
- It is same as HashMap instead maintains ascending order.

TreeMap class declaration

Let's see the declaration for java.util.TreeMap class.

1. **public class** TreeMap<K,V> **extends** AbstractMap<K,V> **implements** NavigableMap<K,V>, Cloneable, Serializable

TreeMap class Parameters

Let's see the Parameters for java.util.TreeMap class.

- **K**: It is the type of keys maintained by this map.
- **V**: It is the type of mapped values.

Constructors of Java TreeMap class

TreeMap()-It is used to construct an empty tree map that will be sorted using the natural order of its key.

TreeMap(Comparator comp)- It is used to construct an empty tree-based map that will be sorted using the comparator **comp**.

TreeMap(Map m)- It is used to initialize a tree map with the entries from **m**, which will be sorted using the natural order of the keys.

TreeMap(SortedMap sm)- It is used to initialize a tree map with the entries from the SortedMap **sm**, which will be sorted in the same order as **sm**.

Methods of Java TreeMap class:

boolean containsKey(Object key)-	It is used to return true if this map contains a mapping for the specified key.
boolean containsValue(Object value)-	It is used to return true if this map maps one or more keys to the specified value.
Object firstKey()-	It is used to return the first (lowest) key currently in this sorted map.
Object get(Object key)-	It is used to return the value to which this map maps the specified key.
Object lastKey()-	It is used to return the last (highest) key currently in this sorted map.
Object remove(Object key)-	It is used to remove the mapping for this key from this TreeMap if present.
void putAll(Map map)-	It is used to copy all of the mappings from the specified map to this map.
Set entrySet()-	It is used to return a set view of the mappings contained in this map.
int size()-	It is used to return the number of key-value mappings in this map.
Collection values()-	It is used to return a collection view of the values contained in this map

Java TreeMap Example:

```
import java.util.*;

class TestCollection15{

    public static void main(String args[]){

        TreeMap<Integer,String> hm=new TreeMap<Integer,String>();

        hm.put(100,"Amit");

        hm.put(102,"Ravi");

        hm.put(101,"Vijay");

        hm.put(103,"Rahul");

        for(Map.Entry m:hm.entrySet()){

            System.out.println(m.getKey()+" "+m.getValue());

        }

    }

}
```

Java TreeMap Example: remove()

```
import java.util.*;

public class TreeMapExample {

    public static void main(String args[]) {

        // Create and populate tree map
```

```

Map<Integer, String> map = new TreeMap<Integer, String>();

map.put(102, "Let us C");

map.put(103, "Operating System");

map.put(101, "Data Communication and Networking");

System.out.println("Values before remove: "+ map);

// Remove value for key 102

map.remove(102);

System.out.println("Values after remove: "+ map);

}

}

```

Generics

A class that can refer to any type is known as generic class.

Generic classes were added in JDK 1.5 version to provide “compile-time type checking” and removing the risk of “ClassCastException” that was common while working with collection classes.

Advantages of Java Generics:

- 1.Type-safety:We can hold a single type of objects in generics.It doesnot allow to store other objects.
- 2.Type casting is not required:There is no need to typecast the objects.Before generics,we need to type cast.

Ex:

```

Class ExOnGenerics{
Public static void main(String[] args){
List<Integer> l=new ArrayList<Integer>();
l.add(10);
l.add(20);
l.add(30);
for(Integer i:l)
{
System.out.println(i);
}
}
}

```

Note:

At the time of list creation,we have specified that the type of elements in the list will be integer.So if we try to add any other type of object in the list,the program will throw compile time error.Also notice that in for loop we don't need type casting of the element in the list,hence removing the ClassCastException at run time.

Ex2:

```

import java.util.*;
class GenericDemo{
public static void main(String[] args){
Map<Integer,String> map=new HashMap<Integer,String>();
map.put(1,"ABC");
map.put(2,"XYZ");
map.put(3,"PQR");
Set <Map.Entry<Integer,String>> set=map.entrySet();
Iterator <Map.Entry<Integer,String>> i=set.iterator();
While(i.hasNext())
{
Map.Entry e=i.next();
System.out.println(e.getKey()+" "+e.getValue());
}
}
}

```

```
}  
}
```

Generic class:

A class that can refer to any type is known as generic class. Here we are using “T” type parameter to create the generic class of specific type.

Creating Generic Class:

```
Class GenDemo<T>{  
    T obj;  
    Void add(T obj){  
        This.obj=obj;  
    }  
    T get(){  
        return obj;  
    }  
}
```

The T type indicates that it can refer to any type (like string, Integer, Employee etc).

The type you specify for the class, will be used to store and retrieve the data.

Ex:

```
Public class Box<T>{  
    Private T t;  
    Public void add(T t)  
    {  
        This.t=t;  
    }  
    Public T get(){  
        Return T;  
    }  
    Public static void main(String[] args){  
        Box<Integer> ibox=new Box<Integer>();  
        Box<String> sbox=new Box<String>();  
        ibox.add(new Integer(10));  
        sbox.add(new String("JNTUK"));  
        System.out.println(ibox.get());  
        System.out.println(sbox.get());  
    }  
}
```

Type Parameters:

The type parameters naming convention are important to learn generics thoroughly. The common type parameters are as follows:

1. T-Type
2. E-Element
3. K-Key
4. N-Number
5. V-Value

Generic Method:

Like generic class, we can create generic method that can accept any type of argument.

Example of java generic method to print array elements, we are using here E to denote the element.

Ex:

```
Class Gen<T>  
{  
    T ob;  
    Gen(T ob){  
        This.ob=ob;  
    }  
    Public void show()  
    {  
        System.out.println("The type of Ob"+Ob.getClass.getName());  
    }  
    Public T getOb(){  
        return ob;  
    }  
}
```

```

}
}
Class GenTest{
Public static void main(Strig[] args)
{
Gen<String> g1=new Gen<String>("durga");
g1.show();
System.out.println(g1.getob());
}
}

```

The above program is an example of generic classes.

Ex on Generic method:

```

Public class GenDemo{
Public static <E> void printArray(E[] elements)
{
for(E element : elements){
system.out.println(element);
}
System.out.println();
}
Public static void main(String[] args)
{
Integer[] intArray={10,20,30,40,50};
Character charArray={'J','N','T','U','K'};
System.out.println("Printing Integer Array");
printArray(intArray);
Systemout.println("Printing character array");
printArray(charArray);
}
}

```

Wildcard in Java Generics:

The ?(question mark) symbol represents wildcard elements.It means any type.

If we write <? Extends number>,it means any child class of number.

Ex:

Integer,float,double,etc.Now we can call the method of Number class through any child class object.

```

Import java.util.*;
abstract class Shape{
abstract void draw();
}
class Rectangle extends Shape{
void draw()
{
Systemout.println("Drawing Rectangle");
}
}
class Circle extends Shape{
void draw(){
System.out.println("Drawing Circle");
}
}
class GenDemo{
public static void drawShapes(List <? Extends Shape> list){
for(Shape s : list){
s.draw();
}
}
public static void main(String[] args){
List<Rectangle> l1=new ArrayList<Rectangle>();

```

```

l1.add(new Rectangle());
List<Circle> l2=new ArrayList<Circle>();
l2.add(new Circle());
drawShapes(l1);
drawShapes(l2);
}
}

```

Wrapper Classes

The main objectives of wrapper classes are:

1.To wrap primitive type into Object form so that we can handle primitives also just like Objects.

Ex:Before JDK 1.5version there is no concept of wrapper class.Upto JDK 1.4 version If we try to add primitives directly to the collection we get compilation error.

```
ArrayList l=new ArrayList();
```

```
l.add(10);//Compile time error
```

```
Integer i=new Integer(10);
```

```
l.add(i);//If we write like this we didn't get error.
```

2.To define several utility methods which are required for the primitive.

```
Ex:String s=Integer.toString(10);
```

How to create Wrapper Object?

Constructors:There are 8 wrapper classes.Almost all wrapper classes contains 2 constructors.

1.Integer:

- Integer i=new Integer(10);//int primitive
- Integer i=new Integer("10");//string

Note:

Almost all wrapper classes contains 2 constructors,one can take corresponding primitive as argument and the other can take string as argument.

2.Double:

- Double d=new Double(10.5);
- Double d=new Double("10.5");

Note:If the string argument not representing a number then we will get an RuntimeException sayng NumberFormartException.

```
Ex:Integer i=new Integer("TEN");
```

3.Char:

- Character ch=new Character('a');

4.Float:

- Float f=new Float(10.5f);//float primitive
- Float f=new Float("10.5");//String

- Float f=new Float(10.5);//double

5.Long:

- long l=new Long(10.34567);//long primitive
- Long l=new Long("10.34");//String

6.Short:

- Short s=new Short(10.3);//Short primitive
- Short s=new Short("10.3");//String

7.Byte:

- Byte b=new Byte(8);//Byte primitive
- Byte b=new Byte("8");

8.Boolean:

- Boolean b=new Boolean(true);//Boolean primitive
- Boolean b=new Boolean("CSE");//String

Summary:

WrapperClass	Corresponding Constructor Argument
Byte	byte or string
Short	short or string
Integer	int or string
Long	long or string
Float	float or string or double
Double	double or string
Character	character
Boolean	boolean or string

Ex:

```
Class Test{
public static void main(String[] args){
Boolean x=new Boolean("yes");
Boolean y=new Boolean("no");
System.out.println(x);
System.out.println(y);
System.out.println(x.equals(y));
}
}
```

Note:In all wrapper classes equals() method is overridden for content comparison.

Methods(Utility Methods) in Wrapper Classes:

1.valueOf():

It is an alternative of constructor. We can use valueOf() method to create wrapper Object for the given primitive or string.

Ex: Integer i=new Integer(10);

or

Integer i=Integer.valueOf(10);

Note: There are multiple forms of valueOf() method

Form1: Every wrapper class except Character class contains a static valueOf() to create wrapper object for the given string.

Syn: public static wrapper valueOf(String s);

Ex:

- Integer i=Integer.valueOf("10");
- Double d=Double.valueOf("10.5");
- Boolean b=new Boolean.valueOf("Durga");//false will be saved

Form2: Every integral type wrapper class(Byte, Short, Integer, Long) contains the following valueOf() to create Wrapper Object for the given specified radix string.

Syn: public static wrapper valueOf(String s, int radix)//radix means if we pass out of bound of given range we get Number format

Ex:

- Integer i=Integer.valueOf("1111");
System.out.println(i);//decimal value
- Integer i=Integer.valueOf("1111",2);
System.out.println(i);//binary value

Allowed range of radix(base) is 2 to 36:

Base 2-0,1
Base 3-0 to 2
Base 8-0 to 7
Base 10-0 to 9
Base 11-0 to 9,a
Base 16-0 to 9,a to f
Base 36-0 to 9,a to z

Form 3: Every wrapper class contains including character class a static `valueOf()` method to create wrapper Object for the given primitive.

Syn: `public static wrapper valueOf(primitive p);`

Ex:

- `Integer i=Integer.valueOf(10);`
- `Double d=Double.valueOf(10.5);`
- `Character c=Character.valueOf('a');`

2.xxxValue():

We can use `xxxValue()` methods to get primitive for the given Wrapper Object.

Note: Every number type wrapper class contains contains the following six methods:

- `public byte byteValue();`
- `public short shortValue();`
- `public int intValue();`
- `public long longValue();`
- `public float floatValue();`
- `public double doubleValue();`

Note:

1.`charValue()` is available in Character class.It is used to get the char primitive for the given Character Object.

Syn: `public char charValue();`

Ex:

- `Character ch=new Character('a');`
`Char c=ch.charValue();`
`System.out.println(c);`

2.`booleanValue()` is available in Boolean class.It is used to get the primitive for the given Boolean Object.

Syn: `public boolean booleanValue();`

Ex:

- `Boolean b=Boolean.valueOf("Durga");`
`boolean b=b.booleanValue();`
`System.out.println(b);`

3.parseXxx():

We can use `parseXxx()` method to convert String to primitive.

Form1: Every Wrapper class except Character class contains the following `parseXxx()` method to find the primitive for the given String object.

Syn: `public static primitive parseXxx(String s);`

Ex:

- `int i=Integer.parseInt("10");`
- `double d=Double.parseDouble("10.5");`

Form2: Every integral type wrapper class(Byte,Short,Int,Long) contains the following `parseXxx()` to convert specified radix string to primitive.

Syn: `public static static primitive parseXxx(string s,int radix);`

Ex:

- `int i=Integer.parseInt("1111",2);` //radix range is 2 to 36
`System.out.println(i);`

4.toString():

We can use `toString()` method to convert wrapper object or primitive to String type.

Syn: `public string toString();`

Ex:

- `Integer i=new Integer(10);`
`String s=i.toString();`
`System.out.println(s);` //10

Form1: Every wrapper class contains the toString() method to convert wrapper Object to String type. It is an overriding version of Object class.

Note: Integer I=new Integer(10);
System.out.println(I);

Internally,

System.out.println(I.toString());

Whenever we are trying to print wrapper Object reference, internally toString() method will be called.

Form2: Every wrapper class including Character class contains the following static toString() method to convert primitive to String.

Syn: public static String toString(primitive p);

Ex:

- String s=Integer.toString(10);
- String s=Boolean.toString(true);
- String s=Character.toString('a');

Form3: Integer and Long classes contain the below toString() method to convert primitive to specified radix string.

Syn: public static String toString(primitive p, int radix); //radix range is 2 to 36.

Ex: String s=Integer.toString(15,2);

System.out.println(s); //1111

Form4: Integer and Long classes contain the following toXxxString() methods:

1. public static String toBinaryString(primitive p)

2. public static String toOctalString(primitive p)

3. public static String toHexString(primitive p)

Ex: String s=Integer.toString(15,2);

The following is the short cut for above method

- String s=Integer.toBinaryString(15);
System.out.println(s);
- String s=Integer.toOctalString(15);
System.out.println(s);
- String s=Integer.toHexString(15);
System.out.println(s);

AutoBoxing and AutoUnBoxing in Java

Autoboxing: Converting java primitive type to object type is called Autoboxing. The concept of autoboxing was introduced from JDK 1.5 version onwards.

Note: Internally autoBoxing concept is implemented by using valueOf() method.

Ex:

```
class ExOnAutoBox{
public static void main(String[] args){
int x=10;
Integer i=x;
System.out.println(i);
}
}
```

Note:

- If we execute the above program by using JDK 1.4 version we get compile time errors (Incompatible types error).
- If we execute the above program by using JDK 1.5 or more we didn't get compile time errors.

AutoUnboxing: Converting java object type into primitive type is called AutoUnboxing.

Note: Internally AutoUnboxing concept is implemented by using xxxValue() method.

Ex:

```
class ExOnAutoUnBox{
public static void main(String[] args){
Integer i=new Integer(10);
int x=i;
System.out.println(x);
}
}
```

Note:

- If we execute the above program by using JDK 1.4 version we get compile time errors(Incompatible types error).
- If we execute the above program by using JDK 1.5 or more we didn't get compile time errors.

Note: Just because of autoBoxing and AutoUnBoxing we can use primitives and wrapper objects interchangeable, from JDK 1.5 version onwards.

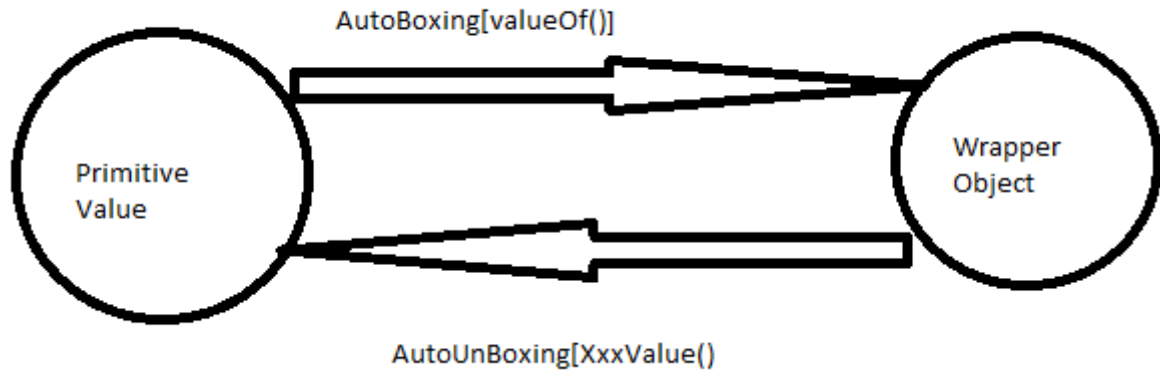


Fig:AutoBoxing and AutoUnboxing

Serialisation and Deserialisation in Java

Serialisation in Java:

- The process of converting Java object into network supporting form is called as serialization.
- Deserialization is the reverse process of serialization.(Converting bits into Java object)

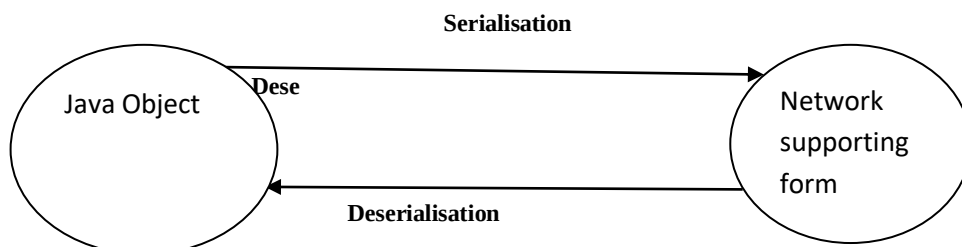


Fig:Representing serialization and deserialisation

Ex on Serialisation:

```

Class Student implement Serializable{
Public static void main(String [] args){
Student s=new Student();
FileOutputStream fos=new FileOutputStream("ABC.txt");
ObjectOutputStream oos=new ObjectOutputStream(fos);
Oos.writeObject(s);
}
}

```

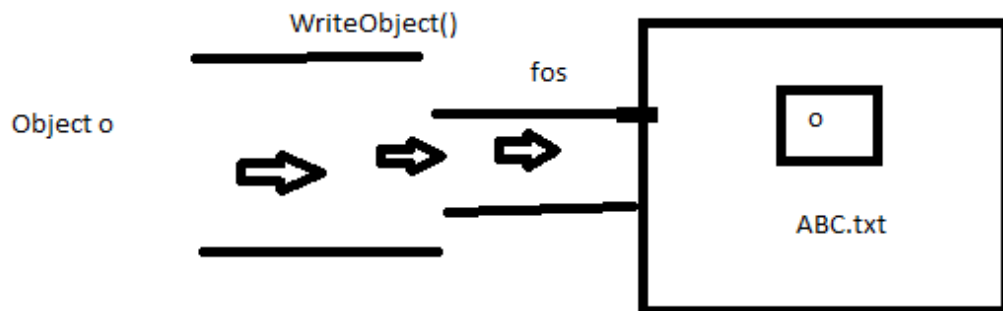


Fig:Serialisation process

Ex on deserialisation:

```

Class Student implement Serializable{
Public static void main(String [] args){
Student s=new Student();
FileInputStream fis=new FileInputStream("ABC.txt");
ObjectInputStream ois=new ObjectInputStream(fis);
Object o=ois.readObject(s);
Syste.out.println(o);
}
}

```

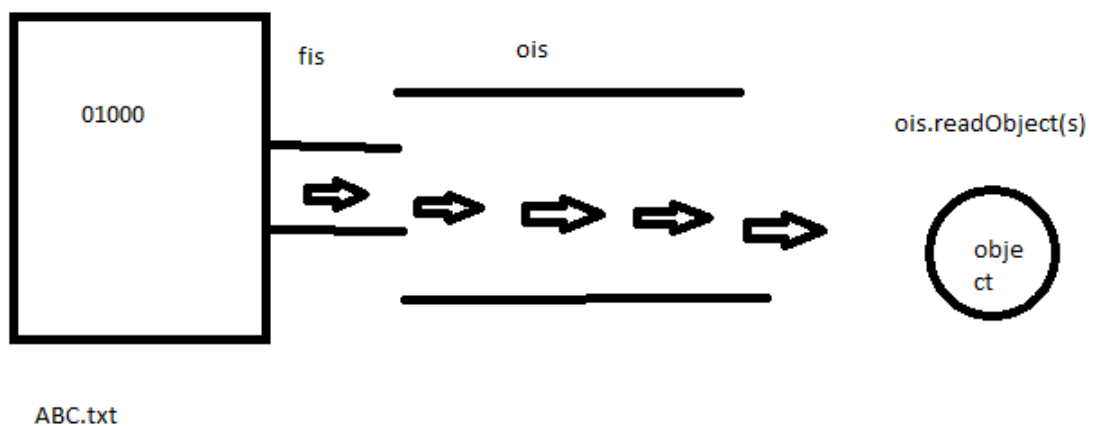


Fig:Deserialisation Process

Java Transient Keyword:

Java transient keyword is used in serialization. If you define any data member as transient, it will not be serialized.

Let's take an example, I have declared a class as Student, it has three data members id, name and age. If you serialize the object, all the values will be serialized but I don't want to serialize one value, e.g. age then we can declare the age data member as transient.

Example of Java Transient Keyword:

In this example, we have created the two classes Student and PersistExample. The age data member of the Student class is declared as transient, its value will not be serialized.

If you deserialize the object, you will get the default value for transient variable.

Let's create a class with transient variable.

```
import java.io.Serializable;
public class Student implements Serializable{
    int id;
    String name;
    transient int age;//Now it will not be serialized
    public Student(int id, String name,int age) {
        this.id = id;
        this.name = name;
        this.age=age;
    }
    public static void main(String args[])throws Exception{
        Student s1 =new Student(211,"ravi",22);//creating object
        //writing object into file
        FileOutputStream f=new FileOutputStream("f.txt");
        ObjectOutputStream out=new ObjectOutputStream(f);
        out.writeObject(s1);
        out.flush();

        out.close();
        f.close();
        System.out.println("success");
    }
}
```